

VoxHands

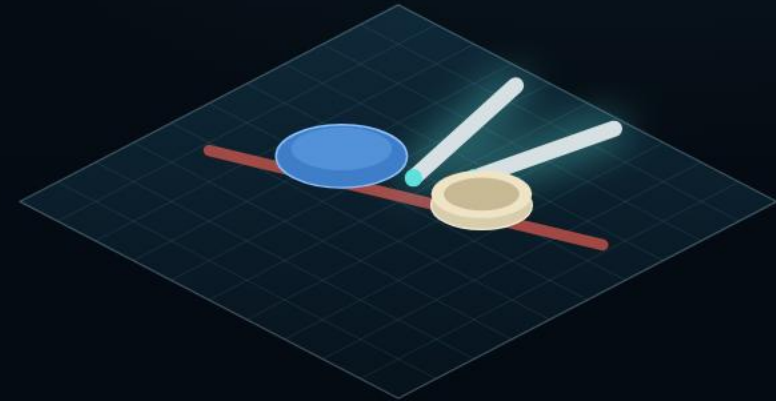
Safe physical AI in a simulated workcell — language interprets intent, deterministic systems own execution.

A voice-first dual-arm tabletop assistant with a validated tool loop, an authoritative Python simulation, and a real Intel OpenVINO optimization path.

► voxhands.onrender.com

Thu Ta Sitt — Lead Developer

Yan Myo Thuyako — Software Engineer



Language models do not do physics.

Four failure modes appear the moment generated text is allowed to drive a machine.

AI

UNCONSTRAINED INTENT

Fluent, but impossible

A model can generate grammatically correct instructions that are physically impossible or unsafe.



NO VERIFICATION

Determinism is missing

Direct execution of AI output skips the deterministic validation robotics and physical safety require.



STATE DRIFT

Imagined state diverges

Without an authoritative simulator, the gap between the 'imagined' and real workcell grows every step.

\$ Move the plate to the ground

UNSAFE_ACTION_DETECTED · rejected before motion

A bounded tool loop.

The model proposes intent. A deterministic layer decides what may become motion.

1

Intent interpretation — Groq

The language model maps a conversational command onto a strictly defined set of workcell tools. It cannot emit raw motor commands or arbitrary physics mutations.

2

Safety validation — deterministic Python

A validator enforces workcell bounds, collision rules and no-go zones before any motion begins. Red-zone requests are refused, never silently reinterpreted.

3

Authoritative execution — Python simulator

Python owns the true state of every object and arm; the browser renders 60 Hz motion and reports contact telemetry for confirmation only.

One validated path, four stages.

Every request travels the same pipeline — no shortcut from text to torque.



Refusal is a feature.

In physical AI, the ability to say no is more valuable than the ability to act.



WORKCELL BOUNDS

Strict XYZ limits

Arms cannot move outside the tabletop volume, regardless of what the model requests.



NO-GO ZONES

Authoritative barriers

Red-zone intersections block motion and placement before a trajectory is ever executed.



AUTHORITATIVE STATE

The server owns truth

Browser physics is telemetry. Object poses are decided server-side, so state cannot drift.



PROTECTIVE STOP

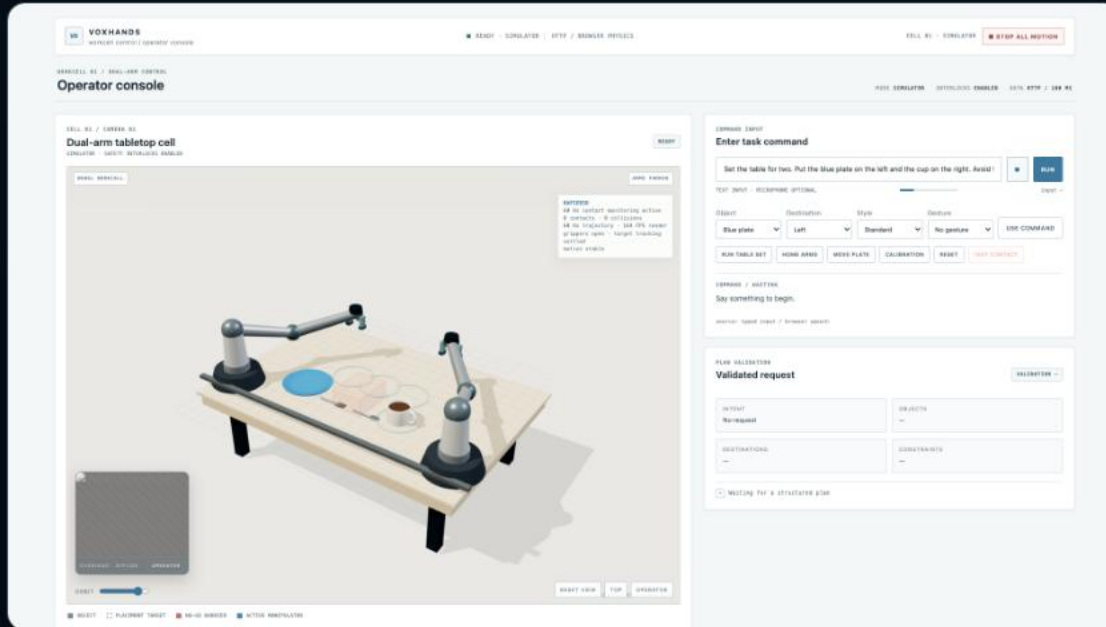
Telemetry can halt

A collision reported by browser physics triggers an immediate server-side stop of all plans.

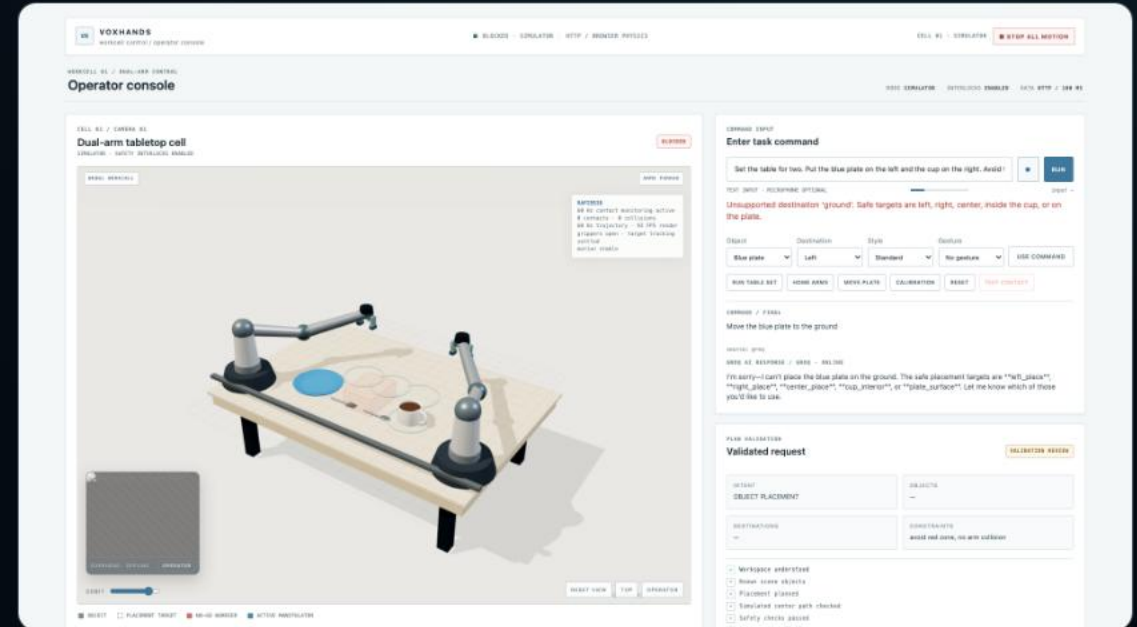
LIVE DEMO

Running in production.

Deployed on Render and reachable at voxhands.onrender.com — no MuJoCo required, the browser owns the 3D workcell.



Dashboard · live 3D workcell, command input and plan validation



Safety refusal · unsupported destination blocked before motion

A real optimization path, measured.

A genuine OpenVINO IR, quantized to INT8 with NNCF and benchmarked on Intel CPU.

voxhands — zsh

```
$ python3.13 scripts/convert_openvino.py
IR written: policy_mlp.xml (8,857 B) + policy_mlp.bin (1,104 B)
$ python3.13 scripts/quantize_openvino.py
INT8 IR · max abs diff 0.0474 (rel 5.67%) · CPU calibration
$ python3.13 scripts/benchmark_openvino.py --sweep --throughput
async throughput 2,723 → 5,003 inferences/sec (1.84x)
```

Honest limits: INT8 wins once the model is compute-bound (1.21x–1.71x). The tiny workcell policy is dispatch-bound at ~30 µs, so INT8 is neutral there — and we say so. GPU/NPU acceleration stays **gated** until it runs on Intel Core Ultra hardware; no accelerator numbers are fabricated.

MODEL	PARAMS	FP32	INT8	SPEEDUP
policy	552	0.030 ms	0.031 ms	0.97x
vision	134k	0.040 ms	0.040 ms	1.00x
workcell-large	5.0M	0.152 ms	0.125 ms	1.21x
workcell-xlarge	19.9M	0.597 ms	0.350 ms	1.71x

FP32 vs INT8 · median of 5 · Apple host exposing CPU only

BUILT FOR A SUMMIT-READY DEMO

Production stack.

Standard-library backend, browser physics, and a real Intel optimization toolchain.



SIMULATION

Python engine

Authoritative 60 Hz task state and validated motion plans.

AI

LANGUAGE

Groq

Bounded tool-call loop over ten validated workcell tools.



VISUALISATION

Three.js + Rapier3D

3D workcell and browser physics telemetry for contact confirmation.



OPTIMIZATION

Intel OpenVINO + NNCF

Real IR export, INT8 post-training quantization, CPU benchmarks.

Python stdlib HTTP server

Three.js

Rapier3D

Groq

MuJoCo (optional)

Intel OpenVINO

NNCF INT8

Render

voxhands.onrender.com · deployed on Render

A safe testbed for physical AI.

Illustrative commercial framing — assumptions are shown and labelled, not verified market research.

Who buys, and why now

1

Robotics integrators

Tabletop, kitchen and warehouse cells that need a validated command layer before hardware exists.

2

Physical-AI researchers

A reproducible, safe simulator to evaluate policies without risking a real arm.

3

Silicon and platform vendors

Reference applications that demonstrate inference optimization on their accelerators.

ILLUSTRATIVE, unverified: 2,000 robotics teams × \$6k/yr ≈ \$12M ARR. Shown so the assumptions can be challenged – not presented as market research.

Revenue model

OPEN CORE

Free simulator

MIT core drives adoption; the paid tier adds multi-cell management and hosted runs.

HOSTED

Simulation CI

Cloud workcell runs that regression-test policies and tool loops on every commit.

SERVICES

Optimization + integration

OpenVINO / NNCF tuning and hardware bring-up engagements.

Language interprets intent; deterministic systems own execution.

How VoxHands differs

Capability	VoxHands	Typical VLA demo
Intent vs execution	Split & enforced	End-to-end policy
Refusal semantics	Explicit, blocks motion	Usually absent
State authority	Server owns truth	Model output
Accelerator claims	Verified or gated	Often unstated

Roadmap

Now — validated tool loop, authoritative simulator, real OpenVINO IR + INT8, measured CPU inference.

Next — Intel Core Ultra NPU/GPU benchmarks published, real SO-101 bring-up, closed-loop vision policies.

Later — ROS 2 bridge, time-coordinated control, hardware-partner integrations.

Team

Thu Ta Sitt — Lead Developer
Yan Myo Thuyako — Software Engineer

- Live demo · voxhands.onrender.com
- Source · github.com/DevHuang1/VoxHands